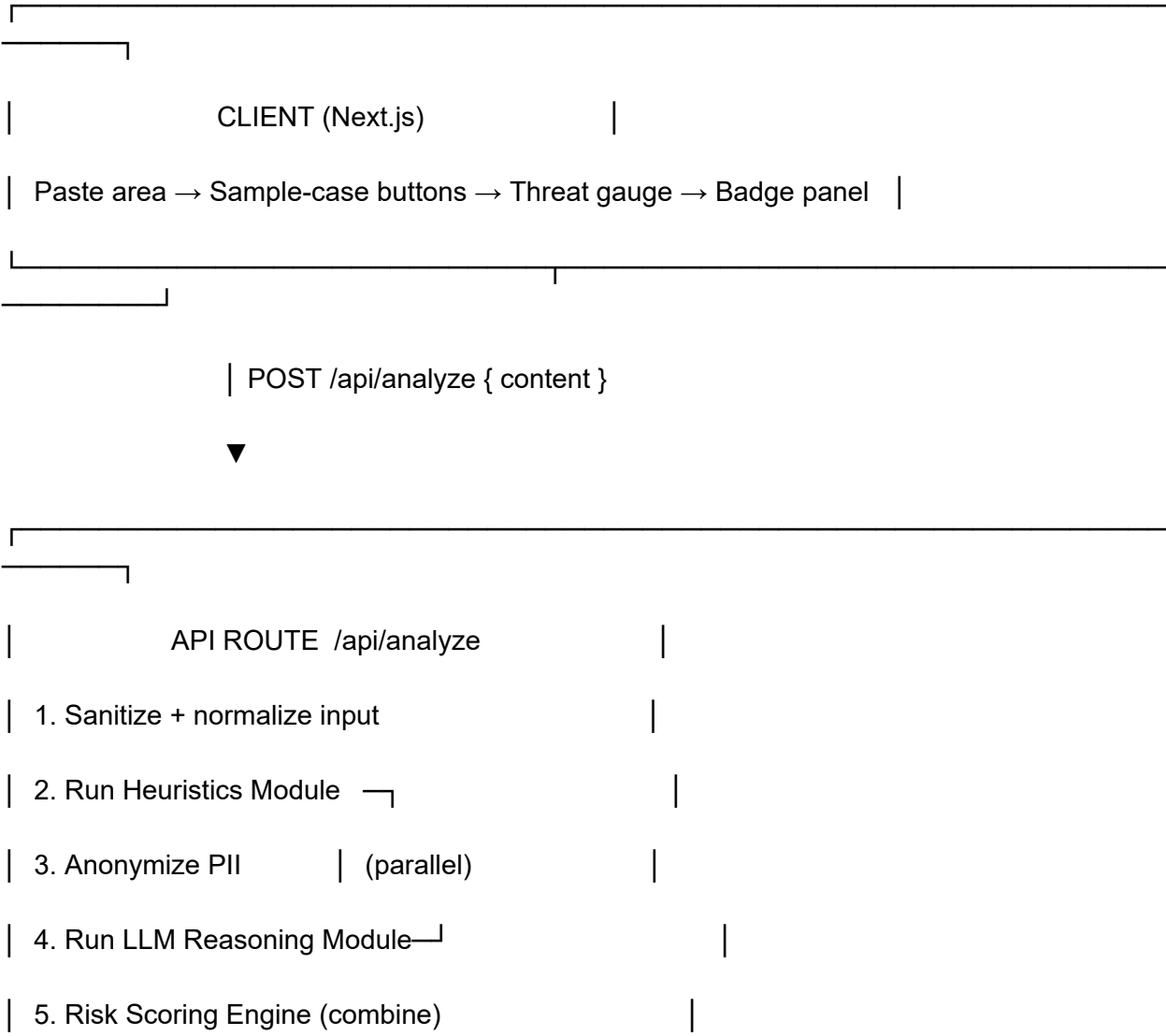


Technical Architecture

AI-Based Phishing and Social Engineering Detection Assistant (PS57)

Companion to [PRD_PhishGuard_PS57.md](#). This document goes one level deeper — module boundaries, data contracts, request flow, and folder structure for the Phase 2 build (Aug 25–29).

1. System Overview



| 6. Return structured JSON response |

Single Next.js app. No separate backend service — API routes under `/app/api/` (or `/pages/api/` depending on Next.js version chosen) handle everything. Keeps Phase 2 build scope realistic for a 4-day sprint.

2. Module Breakdown

2.1 Input Normalizer

Responsibility: Clean and standardize raw input before any analysis.

- Strip whitespace, normalize line endings.
- Extract URLs from free text (regex-based).
- Extract `<a>` tags if input contains HTML (for email-source pastes).
- Lowercase domains for comparison; separate domain from path/query.

Input: raw string (text / SMS / pasted HTML) **Output:**

```
{  
  
  rawText: string,  
  
  extractedUrls: string[],  
  
  anchorTags: { displayText: string, href: string }[]  
}
```

2.2 Heuristics Engine

Responsibility: Fast, deterministic, explainable local checks. No external calls.

Sub-check	Logic	Output
Domain lookalike	Normalized Levenshtein distance against a protected-brands list	<code>{ matchedBrand, distance, suspicious: bool }</code>
Anchor mismatch	Compare <code>displayText</code> vs actual <code>href</code> domain	<code>{ mismatched: bool, displayDomain, hrefDomain }</code>
TLD risk	Static lookup table of high-risk TLDs	<code>{ tld, riskWeight }</code>
Urgency/fear keywords	Keyword/phrase bank match (case-insensitive)	<code>{ matchedPhrases: string[] }</code>

Aggregation: each sub-check contributes a normalized 0–1 sub-score; combined into `heuristicScore` (0–1) via fixed weights (tune during testing, don't hardcode as final — this is a tunable config, not a magic number).

```
// heuristics/index.ts (shape only)
```

```
function runHeuristics(input: NormalizedInput): {
  heuristicScore: number, // 0-1
  signals: {
    domainLookalike?: { matchedBrand: string, distance: number },
    anchorMismatch?: boolean,
    riskyTld?: string,
    urgencyKeywords?: string[]
  }
}
```

2.3 PII Anonymizer

Responsibility: Strip/mask personal identifiers before anything is sent to the LLM API.

- Regex-based redaction: names (if pattern-detectable), phone numbers, email addresses, account numbers.
- Runs **after** heuristics (which need the raw content) but **before** the LLM call.
- This is a privacy commitment stated in the PRD — treat it as non-negotiable in the build, not a stretch goal.

2.4 LLM Reasoning Module

Responsibility: Intent extraction and psychological-pressure analysis — the layer that catches attacks with no malicious link (Case 2 scenario).

System prompt contract (strict JSON, no prose):

```
{
  "risk_score": 0-100,
  "verdict": "Phishing" | "Social Engineering" | "Legitimate" | "Suspicious",
  "threat_factors": ["string", "..."],
  "explanation": "one or two plain-language sentences"
}
```

Design notes:

- Use a low temperature setting for consistency across repeated runs of the same input.
- Validate the response is parseable JSON before returning it to the client; have a fallback verdict if parsing fails (don't let a malformed LLM response crash the demo).
- Send the **anonymized** content only, never the raw PII-containing text.

2.5 Risk Scoring Engine

Responsibility: Combine heuristic and LLM signals into one final score.

$$\text{finalScore} = (0.4 * \text{heuristicScore} * 100) + (0.6 * \text{llmRiskScore})$$

- Weights are a starting point from the PRD — during Phase 2 testing, run all 3 test cases and adjust weights if either signal is over/under-weighted.
- If the LLM call fails (timeout/API error), fall back to heuristic-only scoring rather than blocking the response — the demo must not go blank on a network hiccup.

2.6 API Response Contract

```
{  
  
  "final_score": 78,  
  
  "verdict": "Phishing",  
  
  "threat_factors": ["High Urgency", "Mismatched Domain"],  
  
  "explanation": "This message pretends to be from State Bank...",  
  
  "source_breakdown": {  
  
    "heuristic_score": 65,  
  
    "llm_score": 85  
  
  }  
  
}
```

The `source_breakdown` field isn't strictly required for the demo UI but is worth keeping — it's a good answer to "how do we know your AI isn't just guessing" if a judge asks.

2.7 Frontend Dashboard

Responsibility: Single-page UI, no routing complexity needed.

- Paste/input area.
- 3 sample-case buttons (hardcoded fallback content — demo must work with flaky Wi-Fi).
- Threat gauge (0–100, color-coded).
- Threat-factor badges.
- Explanation text panel.

3. Suggested Folder Structure

/app

/api

/analyze

route.ts → orchestrates the full pipeline

/components

Gauge.tsx

ThreatBadges.tsx

InputPanel.tsx

SampleCaseButtons.tsx

/lib

heuristics/

levenshtein.ts

anchorMismatch.ts

tldRisk.ts

urgencyKeywords.ts

index.ts → runHeuristics()

llm/

prompt.ts → system prompt template

client.ts → API call wrapper

piiAnonymizer.ts

scoringEngine.ts

protectedBrands.ts → reference domain list

page.tsx

Keeping heuristics as isolated, individually testable functions matters — during the live demo, if someone asks "how does domain detection work," you want to point at one small file, not a tangled monolith.

4. Data Flow (Sequence)

1. User pastes content or clicks a sample-case button.
 2. Client sends `POST /api/analyze` with raw content.
 3. API route: Input Normalizer → Heuristics Engine (parallel with PII Anonymizer prep).
 4. PII Anonymizer masks sensitive fields.
 5. Anonymized content sent to LLM Reasoning Module.
 6. Risk Scoring Engine combines heuristic + LLM scores.
 7. JSON response returned to client.
 8. Client renders gauge, badges, and explanation.
-

5. Failure Modes to Handle (don't skip these — judges probe here)

Failure	Handling
LLM API timeout/error	Fall back to heuristic-only score; show a "AI analysis unavailable" note, not a crash
Malformed LLM JSON response	Retry once, then fall back to a generic "Suspicious — manual review recommended" verdict
No network during live demo	Sample-case buttons must use hardcoded pre-computed responses, not live API calls
Legitimate message flagged (false positive)	Weighted scoring, not binary AND logic, so one aggressive signal doesn't dominate

6. What's Explicitly Out of Scope (Phase 2)

- No database/persistence layer — in-memory/session only.

- No user auth.
- No real inbox/email-client integration (that's Future Scope in the PRD).
- No homoglyph/Unicode confusable detection (documented limitation, not a Phase 2 build item).